

Gem5-AcceSys: Enabling System-Level Exploration of Standard Interconnects for Novel Accelerators

Qunyou Liu*, Marina Zapater†, David Atienza*,

**Embedded Systems Laboratory (ESL), EPFL, Switzerland, qunyou.liu@epfl.ch, david.atienza@epfl.ch*

†*University of Applied Sciences and Arts Western Switzerland (HES-SO), Switzerland, marina.zapater@heig-vd.ch*

Abstract—The growing demand for efficient, high-performance processing in machine learning (ML) and image processing has made hardware accelerators, such as GPUs and Data Streaming Accelerators (DSAs), increasingly essential. These accelerators enhance ML and image processing tasks by offloading computation from the CPU to dedicated hardware. These accelerators rely on interconnects for efficient data transfer, making interconnect design crucial for system-level performance. This paper introduces Gem5-AcceSys, an innovative framework for system-level exploration of standard interconnects and configurable memory hierarchies. Using a matrix multiplication accelerator tailored for transformer workloads as a case study, we evaluate PCIe performance across diverse memory types (DDR4, DDR5, GDDR6, HBM2) and configurations, including host-side and device-side memory. Our findings demonstrate that optimized interconnects can achieve up to 80% of device-side memory performance and, in some scenarios, even surpass it. These results offer actionable insights for system architects, enabling a balanced approach to performance and cost in next-generation accelerator design.

Index Terms—Memory Hierarchy, PCIe, Interconnects, Hardware Accelerators, System-Level Simulation

I. INTRODUCTION

Transformer models, such as Vision Transformer (ViT) [1], have revolutionized machine learning (ML) and natural language processing (NLP), setting new benchmarks in various tasks [2]. Their growing adoption in real-time applications demands efficient and scalable hardware architectures to meet rising computational needs. Systolic arrays have shown promise in accelerating these tasks, offering parallel data flow and high computational throughput ideal for matrix multiplication, central to both traditional ML and transformer-based models, such as Google’s Tensor Processing Unit [3] and TiC-SAT [4]. However, most studies on systolic-array-based architectures primarily focus on hardware-level evaluations, often neglecting the broader system-level context. This leaves a gap in understanding how the interconnects and memory hierarchy, which are vital components in moving data between processors and memory, impact the overall system performance. While some system-level evaluations exist, they often lack the use of standard interconnects such as PCIe, and heterogeneous memory hierarchy such as non-uniform memory access (NUMA), reducing their practicality for real-world applications.

To bridge this gap, we introduce Gem5-AcceSys, a novel framework enabling system-level exploration of interconnects and memory hierarchies in accelerators. Using a matrix multiplication accelerator tailored for transformer workloads as a case study, our design framework incorporates essential components such as PCIe [5], [6], NUMA architectures, and configurable memory hierarchies. This integration provides a

practical platform for evaluating real-world performance under various configurations. Specifically, we assess the impact of standard interconnects (PCIe) and diverse memory types, including DDR3/4/5, GDDR6, and HBM2, on system efficiency. By emphasizing system-level architecture, our work provides a comprehensive understanding of how interconnects and memory hierarchies influence transformer’s performance and how general matrix multiplication (GEMM) and Non-GEMM impact the system performance with the overhead introduced by standard interconnects. This research offers actionable insights for designing scalable and cost-effective accelerator systems tailored to modern ML workloads. More specifically, our contributions are as follows:

- 1) **Framework for Interconnect Exploration:** We present the framework, Gem5-AcceSys, to support PCIe interconnects, NUMA architecture, and processing near memory based on the Gem5 simulator, enabling system-level co-design with features like device-side memory, local buffer, SMMU, DMA, and RTL-based accelerators.
- 2) **PCIe Bandwidth impact on Performance Study:** We investigate the impact of varying PCIe bandwidth by adjusting the number of lanes and lane speeds for the target GEMM workload. Additionally, we vary the packet request size from the perspective of the accelerator to identify the optimal packet size for the design.
- 3) **Memory Access and Address Translation Study:** We analyze the impact of memory type and location on system performance from a system-level perspective. Additionally, we investigate address translation overhead for Transformer workloads, demonstrating the effects of virtual address spaces on accelerator efficiency.
- 4) **Impact of Non-GEMM and GEMM Workload Analysis:** We conduct a detailed runtime analysis of Transformer workloads, dividing them into two components: GEMM and Non-GEMM. To optimize performance, we profiled these workloads separately and identified thresholds to determine when device-side memory should be utilized.

II. STATE-OF-THE-ART

Several open-source simulators have been developed to aid in the design and integration of specialized hardware accelerators, notably Gem5-Aladdin [7], Gem5-Salam [8], Gem5-RTL [9], and Gem5-X [10]. These frameworks provide valuable tools for early-stage design exploration and detailed evaluations of accelerators within system simulations. Gem5-Aladdin integrates accelerator modeling with system simulation, enabling pre-RTL design analysis. Gem5-Salam employs LLVM-based

methodologies to model and estimate the performance and area of custom accelerators. Gem5-RTL incorporates RTL designs for detailed performance assessments, while Gem5-X supports simulations of many-core heterogeneous systems with advanced memory technologies. However, these simulators have limitations that hinder their applicability for comprehensive system-level co-design. They rely on simplistic interconnects (e.g., basic buses, shared memory), lacking support for standard interfaces like PCIe. Memory hierarchy support is limited, with features like Non-Uniform Memory Access (NUMA) and processing near memory (PNM) absent [11]. High simulation overhead due to RTL integration, as seen in frameworks like Gem5-RTL, impedes rapid prototyping. Scalability issues arise when accurately simulating diverse accelerators within large-scale systems. Additionally, inadequate system integration is evident from the lack of features like Direct Memory Access (DMA) and System Memory Management Unit (SMMU) for efficient data handling [12], [13]. These limitations highlight the need for a comprehensive framework that supports standard interconnects, complex memory hierarchies, and realistic full-system interactions. To address this gap, we propose **Gem5-AcceSys** to enable detailed system-level co-design of accelerators.

III. DESIGN FRAMEWORK

A. Overall Architecture and Interface Integration

Fig. 1 illustrates our enhanced design framework, with newly added components highlighted in light blue, significantly expanding the Gem5 simulator's capabilities for contemporary system design. This framework integrates essential functionalities, including PCIe interconnects, DMA, SMMU, configurable sub-memory architecture, and RTL-based accelerator support. The system is organized into two primary sections: the CPU cluster with its cache and the accelerator system interfaced with the memory bus (MemBus) through PCIe components.

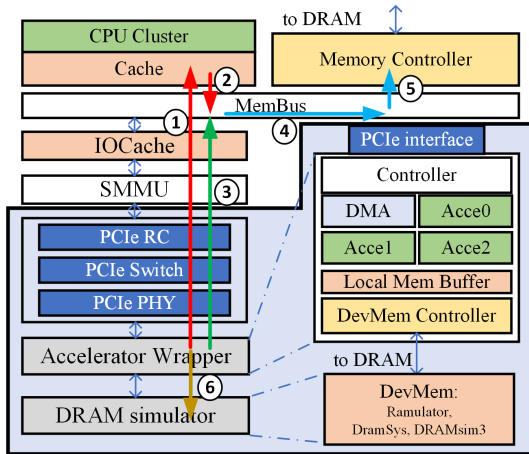


Fig. 1. Design Framework Architecture

The CPU cluster, comprising one or more CPUs and associated caches, connects to the main memory (DRAM) via a memory controller. Data transfers between the CPU and memory are facilitated by a MemBus. PCIe integration provides a realistic interface for peripheral device simulations, surpassing the limitations of conventional memory bus latency models. The DMA feature further enhances system efficiency by enabling

direct memory transfers, reducing the data movement burden on the accelerator and simplifying its design. The SMMU enables virtual-to-physical address translation, streamlining driver development and improving memory mapping efficiency for accelerators [12], [13]. A configurable sub-memory system supports the optimization of memory configurations crucial for co-designing accelerator systems. This includes device-side memory with local buffers and host-side memory configurations to evaluate different system trade-offs. Key PCIe components include:

- **PCIe RC (Root Complex):** The CPU endpoint that manages data and commands on the PCIe bus.
- **PCIe PHY (Physical Layer):** Connects the RC to devices, facilitating communication via the PCIe Switch and Link.
- **PCIe Switch:** Routes traffic among PCIe devices, supporting multiple connections and enhancing scalability.

By positioning the SMMU between the MemBus and PCIe components, the framework enhances memory protection and address translation capabilities, enabling efficient use of virtual address spaces.

B. Accelerator Wrapper

The Accelerator Wrapper contains logic and interfaces for the hardware accelerator, including a PCIe interface that connects to the accelerator's controller. This controller manages data flows between the accelerator and system. Adjacent to this, the single accelerator or accelerator cluster, either RTL-based or C++-based, integrates with the system using Verilator [14] to convert RTL code to C++, compiling it into an executable that runs as a child process with shared memory system calls. A DMA block within the controller enables direct memory access, bypassing the CPU for higher performance. A Local Mem Buffer provides quick, temporary computation storage, and a Device Memory (DevMem) Controller oversees data transfers between the accelerator and device memory, boosting efficiency. The accelerator also interfaces with a configurable memory hierarchy.

C. Memory Hierarchy and Subsystem Performance Analysis

Our design framework incorporates a configurable memory hierarchy, including host-side and device-side memories, along with a last-level cache and a device-side cache, supporting comprehensive design space exploration. It interfaces with DRAM models like DRAMsim3 [15], Ramulator2 [16], and Dramsys5 [17] for accurate DRAM timing and power statistics. The framework offers three memory access methods: direct cache (DC) access, direct memory (DM) access and DevMem access. Fig. 1 illustrates DC and DM modes. In DC mode, data requests from the accelerator are directed to the cache hierarchy. Cache hits retrieve data immediately, reducing latency, while cache misses access host-side memory via the MemBus (arrow 2) and memory controller (arrows 4 and 5), adding latency. This mode enables detailed analysis of cache effects on performance and allows adjustments to cache size and latency. To support this, we modified the simulator to implement a cache coherency model between the accelerator's cache and the CPU cache. In DM mode, requests bypass the cache and go straight to main memory via arrows 3 and 5, minimizing latency by eliminating

cache checks. This method requires software management of data coherency since it bypasses the cache system. Arrows 6 bypasses the whole PCIe system to access device-side memory avoid the data movement overhead introduce by PCIe system. By leveraging configurable memory hierarchies and flexible access methods, our design framework enables the selection of optimal memory configurations to enhance accelerator performance. Additional insights into the impact of memory hierarchy on performance are discussed in Section V.

D. Comparison with Existing Simulators

To highlight the unique features of Gem5-AcceSys and how it extends beyond existing Gem5-based simulators, we provide a concise comparison in Table I. This table summarizes key features such as accelerator (Acce) design level support, interconnect capabilities, address translation, memory simulation, kernel driver support, DMA capabilities, device-side memory support, simulation scope, and accelerator process models.

As shown in Table I, Gem5-AcceSys provides comprehensive support for features essential in modern accelerator design, addressing limitations present in existing simulators. This includes support for standard interconnects like PCIe, accelerator address translation via SMMU, integration with external memory simulators, kernel driver support, multi-channel DMA, and device-side memory.

IV. EXPERIMENTAL SETUP

To demonstrate the capabilities of our design framework and analyze the impact of standard interconnects and configurable memory architectures on transformer workloads, we conduct experiments using the DC access method, with configurations and parameters detailed in Table II.

A. Hardware Accelerator Design

We use a Systolic Array (SA) accelerator named MatrixFlow [18]. MatrixFlow is a loosely-coupled SA specifically optimized for transformer models. It uses a new matrix multiplication technique alongside an optimized data structure, which significantly enhances data streaming efficiency and reduces memory overhead. MatrixFlow contains 16x16 multiply-accumulate units, and uses data in integer format as input and output. Through a combination of hardware-software co-design, including the integration of PCIe and DMA for efficient data handling, MatrixFlow achieves remarkable speed-ups of up to 400x compared to a single-threaded CPU system, and demonstrates substantial performance improvements over both loosely- and tightly-coupled accelerators in the state-of-the-art.

B. Workload

First, we select GEMM as a workload because it represents a general and widely used computational pattern, showcasing the versatility of our design framework in evaluating performance and analyzing bottlenecks. Next, to further illustrate our framework's capabilities in co-optimizing complex systems, we use the Vision Transformer (ViT) [1] as a case study. We evaluate workloads from the ViT_base, ViT_large, and ViT_huge models, which have hidden dimensions of 768, 1024, and 1280, respectively, and utilize 12 or 16 attention heads.

V. PERFORMANCE EVALUATION

To evaluate the performance of our design framework, we focus first on GEMMs and analyze its performance under various configurations. Based on the insights gained from this analysis, we narrow-down the set of interesting configurations and focus on the entire transformer.

A. Performance Bounds and Roofline Model

We begin by analyzing the roofline model of our accelerator system, focusing on GEMMs with dimensions 1024. To isolate the effects of computation on performance, we fix the PCIe bandwidth at 8 GB/s and vary the computation time of the systolic array within the Gem5 framework.

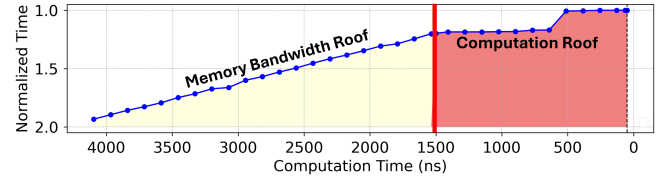


Fig. 2. Roofline Model of the Accelerator System

Fig. 2 illustrates the roofline model of the proposed accelerator. The x-axis represents computation time (ns), while the y-axis shows the normalized execution time. For computation times exceeding 1500 ns, the system operates in the **memory-bound** region, where performance improves linearly as computation time decreases. This indicates that performance is limited by memory bandwidth or PCIe data transfer speed. As computation time drops below 1500 ns, the system transitions to the **computation-bound** region, as shown by the plateau in performance. Here, further improvements in memory bandwidth have no significant effect because the computation speed of the systolic array becomes the bottleneck. This transition, marked by the red line, highlights the shift in limiting factors. To better understand these performance bounds, we proceed with a detailed analysis of GEMM workloads.

B. GEMMs

We start by analyzing the impact of the following factors:

- **PCIe Link:** Analyzing how PCIe bandwidth, latency, and packet sizes affect the data transfer efficiency and overall computation speed.
- **Memory Type, location, latency and bandwidth:** Assessing performance variations between device-side and host-side memory, under different memory technologies including DDR4, LPDDR, GDDR, and HBM.
- **Address Translation:** Analysing the overhead of Address Translation wrt matrix size and their effects on computational delays and resource utilization.

1) PCIe Link Performance Analysis:

a) **Bandwidth Sweeping:** To evaluate the impact of PCIe bandwidth on GEMM performance, we vary the number of lanes (2, 4, 8, 16) and their speeds (2 Gbps to 64 Gbps), using 2048x2048 matrices on the systolic array accelerator. As depicted in Fig. 3, increasing bandwidth consistently reduces execution time, with performance scaling until the system transitions from memory-bound to compute-bound at 16 lanes. Notably, bandwidth impacts performance significantly, with the highest bandwidth outperforming the lowest by up to

TABLE I
COMPARISON OF GEM5-BASED FRAMEWORKS FOR HARDWARE ACCELERATOR SIMULATION

Feature	Gem5-Aladdin	Gem5-SALAM	Gem5-RTL	Gem5-X	Gem5-AcceSys
Acce Design Level	C++	LLVM IR	RTL	C++	C++, RTL
Interconnect	Basic buses	Basic buses	Basic buses	Basic buses	Basic buses, PCIe
Acce Address Translation	Yes	No	No	No	Yes (SMMU)
External Memory simulator	No	No	No	No	Ramulator/DRAMsys
Kernel Driver Support	No	No	No	Limited	Yes
Multi-Channel DMA	Yes	No	No	No	Yes
Device-Side Memory	No	No	No	Yes	Yes
Full-System Simulation	Yes	Bare-metal	Yes	Yes	Yes
Acce Process Model	Integrated	Integrated	Integrated	Integrated	Child process (Multi-threaded)

TABLE II
SYSTEM CONFIGURATION

Component	Specification
CPU	ARM, 1 GHz
Data Cache	64 kB
Instruction Cache	32 kB
Last Level Cache	2 MB
IOCache	32 kB
Memory	DDR3_1600_8x8, 4 GB
PCIe Link	Version 2.0, 4 Gb/s, 4 Lanes
PCIe RootComplex	150ns Latency
PCIe Switch	50ns Latency

1109.9%. This highlights the effectiveness of our framework in identifying optimal bandwidth configurations for balanced performance and resource efficiency.

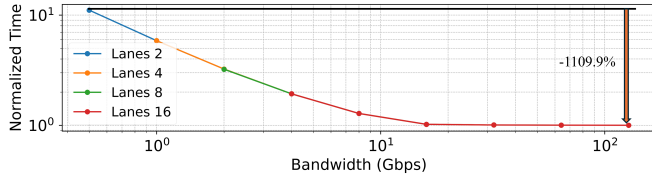


Fig. 3. Performance (Execution time) for Matrix Size 2048 under varying per-lane bandwidth and number of lanes

Key Takeaway #1: PCIe bandwidth significantly impacts accelerator performance, particularly in memory-bound regions. However, as systems become compute-bound, the benefits of additional bandwidth diminish.

b) Packet Size Sweeping: We configure the PCIe Link at 4GB/s, 8GB/s, 16GB/s, 32GB/s, and 64GB/s, analyzing the impact of request packet sizes from accelerator ranging from 64 bytes to 4096 bytes. As shown in Fig. 4, varying packet sizes result in differing processing times, even reach 36%. Initially, execution time decreases as packet size increases from 64 bytes, reaching a minimum around 256 bytes, reflecting efficiency gains at moderate sizes. Beyond this point, execution time increases with larger packet sizes up to 4096 bytes, forming a convex curve that highlights a non-linear relationship: both very small and very large packet sizes are less efficient. This behavior stems from the PCIe hierarchy, where packets must pass through the RootComplex and Switch. Larger packets disrupt the pipeline, causing stalls at each component before data reaches the accelerator.

Key Takeaway #2: Packet size significantly affects execution time with a convex trend: 64-byte packets incur 12% overhead, and 4096-byte packets 36%, relative to the optimal 256-byte size.

2) Memory performance analysis: To examine the performance impact of different memory types and locations, we

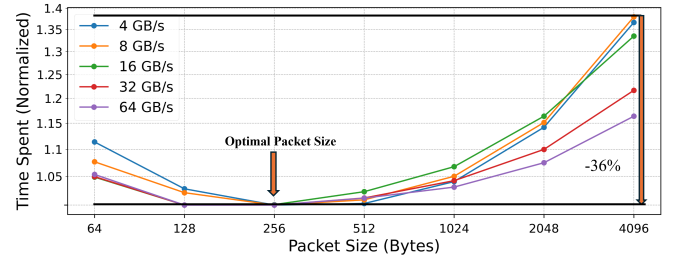


Fig. 4. Execution Time under different packet sizes for different PCIe bandwidth.

use ramulator2 as our backend DRAM model, and test DDR3, DDR4, HBM, and GDDR5 with the bandwidth and data rate configurations detailed in Table III.

TABLE III
MEMORY CONFIGURATION

Component	Channel	Data width	Bandwidth	Data Rate
DDR3	1	64	12.8 GB/s	1600 MT/s
DDR4	1	64	19.2 GB/s	2400 MT/s
DDR5	2	32	25.6 GB/s	3200 MT/s
HBM2	2	128	64 GB/s	2000 MT/s
GDDR6	2	64	32 GB/s	2000 MT/s

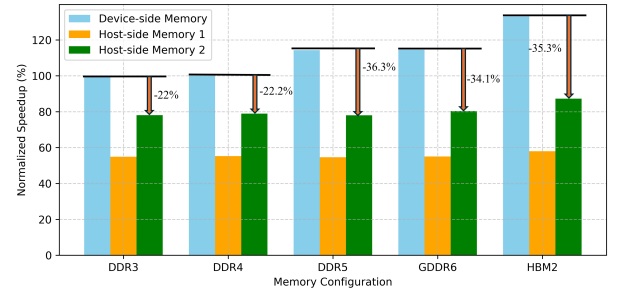


Fig. 5. Impact of DRAM type and location

a) Device-side vs Host-side Memory and Memory Type:

Fig. 5 presents the normalized speedup (wrt DDR4 device-side data) comparison between device-side memory and two host-side memory configurations (one with a 2GB/s and one with a 64GB/s PCIe link) for DDR4, HBM, GDDR5, and LPDDR5.

The results demonstrate that device-side memory consistently outperforms host-side memory across all tested memory types, regardless of the speed of the PCIe bus. Host-side memory shows lower speedups, with a clear dependence on PCIe speed. When using the 64GB/s PCIe configuration, host-side memory can achieve around 78% of the performance relative to device-side memory, which is a substantial improvement over the 2 GB/s PCIe. Performance degradation depends on the memory

technology, and is aggravated for GDDR5 and HBM, as the gap between host-side and device-side memory grows larger.

Key Takeaway #3: Implementing device-side memory significantly boosts performance, improving accelerator efficiency by up to 2 times compared to all other configurations.

b) *Memory bandwidth and latency sweeping:* We investigate the impact of memory bandwidth and latency by varying one while keeping the other one constant, using HBM as a case study with gem5's default DRAM model.

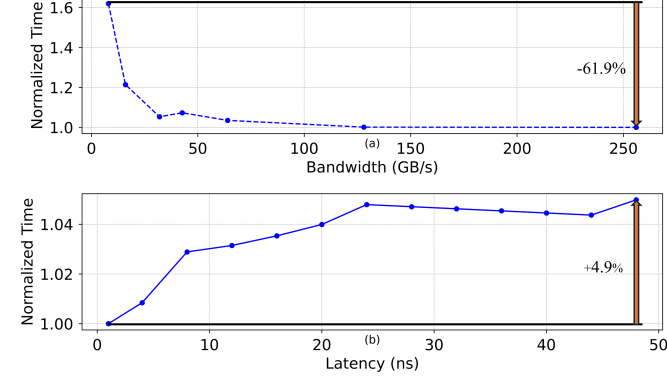


Fig. 6. Impact of Memory Bandwidth(a) and Memory Latency(b)

Fig. 6(a) shows that normalized execution time decreases significantly as bandwidth increases up to approximately 50GB/s, yielding up to a 60% performance improvement. This underscores the critical role of bandwidth in alleviating performance bottlenecks at lower levels. Beyond 100GB/s, the curve plateaus, with only a 1.7% improvement from 50GB/s to 256GB/s, thus other system components become the bottleneck.

Fig. 6(b) illustrates that execution time increases with latency, especially sharply from 1ns to 12ns, then plateaus between 12ns and 36ns. However, the overall time overhead due to increased latency is only about 4.9%. This analysis indicates that while both bandwidth and latency affect performance, the system is significantly more sensitive to changes in bandwidth. Thus, bandwidth is the more crucial factor in GEMM-dominated workloads.

Key Takeaway #4: Bandwidth has a greater impact than latency in GEMM performance: bandwidth improves performance by 60% and latency only adds 5% overhead.

3) *Address Translation performance analysis:* In this section, we analyze the impact of virtual address translation on system performance, focusing on larger matrix sizes. Table IV summarizes key metrics such as memory footprint, translation times, and page table walk (PTW) times.

As matrix size increases, translation overhead initially decreases, reaching 1.00% at 1024, but rises to 6.49% at 2048. This reflects how larger matrices reduce per-translation costs through amortization but face increasing complexity with larger address spaces. Similarly, the mean translation time drops to its lowest value at 1024 but spikes to 54.38 cycles at 2048, indicating inefficiencies. PTW times also increase with matrix size, reaching 368.14 cycles at 2048, highlighting the growing complexity of address translation for very large datasets. This highlights the critical need for careful optimization of address

translation mechanisms to maintain system performance, particularly for large-scale workloads.

Key Takeaway #5: Address translation overhead significantly impacts performance for large memory footprints. Larger matrices increase translation complexity and page table walk times, requiring optimization to avoid bottlenecks.

C. Transformer Performance Evaluation

We apply the GEMM analysis insights to Transformer inference using four system configurations:

- 1) A system using host memory and small PCIe bandwidth of 2 GB/s with 4 4Gbps lanes (PCIe-2GB).
- 2) A system using host memory and moderate PCIe bandwidth of 8 GB/s with 8 8Gbps lanes (PCIe-8GB).
- 3) A system using host memory and PCIe bandwidth of 64 GB/s with 16 64Gbps lanes (PCIe-64GB).
- 4) A system without host memory, utilizing device-side memory instead (DevMem).

We select ViT as the target workload and configure each system based on the GEMM analysis:

- For PCIe-2GB and PCIe-8GB: packet size of 256 and DDR4 memory.
- For PCIe-64GB: packet size of 256 and HBM2 memory.
- For DevMem: packet size of 64 and HBM2 memory.

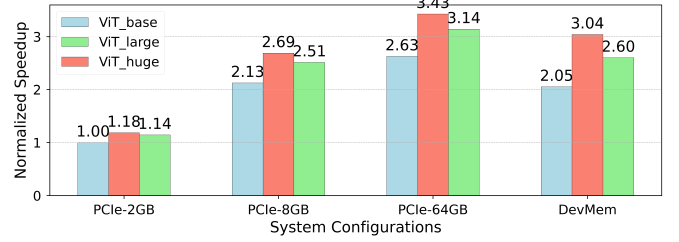


Fig. 7. Performance comparison of memory locations and interconnects

Fig. 7 shows that PCIe-64GB achieves significant performance improvements over the baseline PCIe-2GB, with speedups ranging from approximately 2.5x to 3.4x, highlighting the critical impact of PCIe bandwidth when using host memory. In contrast, the DevMem configuration, despite leveraging device-side memory (HBM2 with burst size of 64) and reduced data transfer times due to proximity to computational units, performs slightly worse than PCIe-64GB.

D. Transformer Workload Analysis

1) *GEMM and Non-GEMM Performance Evaluation:* To understand the performance discrepancies observed, we conduct a detailed analysis of the Transformer workload, profiling both GEMM and Non-GEMM operations across different ViT models and system configurations, as highlighted in prior research [19], [20]. Fig. 8 shows that while DevMem offers the best performance for GEMM workloads due to its high data bandwidth, it exhibits the poorest performance in Non-GEMM operations, incurring up to a 500% overhead compared to systems using the PCIe interface. This degradation stems from the high latency introduced by the NUMA architecture and longer access times to device-side memory. Consequently,

TABLE IV
RESULTS WITH LARGER MATRIX SIZES

Metric	64	128	256	512	1024	2048
Memory Footprint (Pages)	12.0	48.0	192.0	768.0	3072.0	12288.0
Translation Times	3130	18470	142738	1082780	8593259	68430699
Trans Mean Time	23.42683	20.37948	13.87159	9.91735	10.478634	54.38005
PTW Times	15	54	227	1034	7675	479244
PTW Mean Time	176.6666	281.90740	265.255507	252.465184	294.609381	368.141137
uTLB Lookup times	2350	17690	137290	1081610	8586250	68423690
uTLB Misses times	195	862	8644	65808	731513	10416279
Trans Overhead	6.02%	1.87%	1.59%	1.30%	1.00%	6.49%

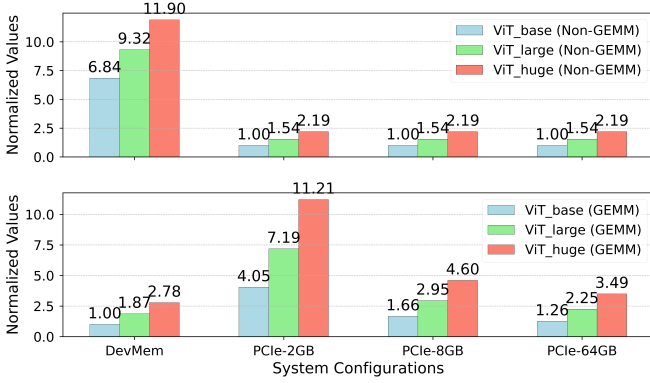


Fig. 8. Performance Comparison of GEMM and Non-GEMM workload

despite DevMem's advantages in GEMM tasks, these overheads hinder system performance in Non-GEMM computations.

Key Takeaway #6: Device-side memory overhead from NUMA architecture can degrade performance, accounting for 40% in Transformer experiments like ViT_large.

2) *Performance Analysis of GEMM and Non-GEMM Workloads:* To quantify the performance trade-offs between GEMM and Non-GEMM workloads, we propose a model that expresses the total execution time of Transformers as a combination of these two components. By understanding their respective contributions and computational requirements, we can determine the optimal balance between device-side and host-side memory utilization. The total execution time $\text{Time}_{\text{overall}}$ of a Transformer workload can be expressed as:

$$\text{Time}_{\text{overall}} = T_{\text{other}} + \frac{W_{\text{GEMM}}}{P_{\text{GEMM}}} + \frac{W_{\text{NonGEMM}}}{P_{\text{NonGEMM}}}$$

Where T_{other} is the fixed time for other operations. W_{GEMM} is the fraction of GEMM workload ($0 \leq W_{\text{GEMM}} \leq 1$). P_{GEMM} and P_{NonGEMM} are the performance metrics for GEMM and Non-GEMM workloads, respectively. In our case, the Non-GEMM percentage represents the proportion of overall time spent on Non-GEMM workloads when executed on a PCIe system configuration.

We vary the W_{NonGEMM} from 0% to 100% to analyze the performance variance. Fig. 9 shows the performance variation with increasing Non-GEMM percentage. Our analysis reveals that DevMem outperforms the PCIe system when the GEMM workload fraction (W_{GEMM}) exceeds certain thresholds, which decrease as PCIe bandwidth increases. Specifically, DevMem is preferable when W_{GEMM} exceeds 34.31% for a PCIe bandwidth

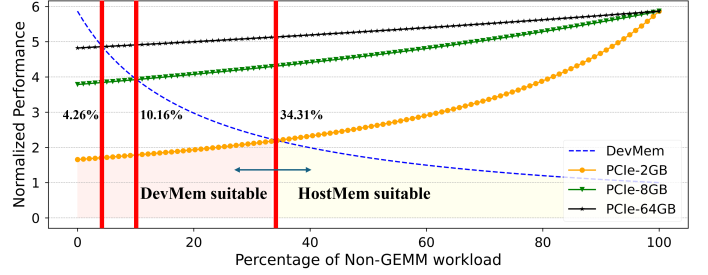


Fig. 9. Overall Transformer Performance as a Function of Non-GEMM Workload Fraction for various PCIe Bandwidths vs DevMem

of 2 GB/s, 10.16% for 8 GB/s, and 4.27% for 64 GB/s. This indicates that as PCIe bandwidth increases, the advantage of using DevMem diminishes unless the workload is overwhelmingly dominated by GEMM operations.

Key Takeaway #7: The choice between PCIe and DevMem depends on workload composition (GEMM vs. Non-GEMM) and PCIe bandwidth; DevMem is preferred when Non-GEMM fractions are below a threshold.

VI. CONCLUSION

In this paper, we have introduced Gem5-AcceSys, a comprehensive framework that extends the Gem5 simulator to support standard interconnects like PCIe, NUMA architectures, and configurable memory hierarchies. This advancement addresses critical limitations of existing simulators, enabling detailed system-level co-design and realistic performance evaluation of hardware accelerators. Using Gem5-AcceSys, we conduct an in-depth analysis of a matrix multiplication accelerator tailored for transformer workloads. Our experiments demonstrate that optimized PCIe interconnects can achieve up to 80% of the performance of systems using device-side memory, and in some cases, even surpass them. We also uncover trade-offs between GEMM and Non-GEMM workloads, highlighting how their balance influences the optimal choice of memory configurations and interconnect strategies. By facilitating the exploration of standard interconnects and memory hierarchies, Gem5-AcceSys provides actionable insights for system architects, empowering them to balance performance and cost in designing efficient and scalable next-generation accelerator systems.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their feedback, Pengbo Yu for hardware and manuscript guidance, and Gabriel Catel Torres and Clément Dieperink for software support.

This work was supported in part by the Swiss State Secretariat for Education, Research, and Innovation (SERI) through the SwissChips research project, and also by Intel as part of the Intel Center for Heterogeneous Integrated Platforms (HIP).

REFERENCES

- [1] A. Dosovitskiy *et al.*, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *Proc. ICLR*, 2021.
- [2] J. Devlin, M. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [3] N. P. Jouppi *et al.*, “In-Datacenter Performance Analysis of a Tensor Processing Unit,” in *Proc. 44th Annu. Int. Symp. on Computer Architecture (ISCA)*, Toronto, ON, Canada, Jun. 2017, pp. 1–12.
- [4] A. Amirshahi, J. A. H. Klein, G. Ansaloni, and D. Aienza, “TiC-SAT: Tightly-Coupled Systolic Accelerator for Transformers,” in *Proc. 28th Asia and South Pacific Design Automation Conf. (ASP-DAC)*, Tokyo, Japan, Jan. 2023, pp. 657–663.
- [5] J. Ajanovic, “PCI Express 3.0 Overview,” in *Proc. 2009 IEEE Hot Chips 21 Symposium*, 2009, pp. 1–61.
- [6] M. Vasa, C.-L. Liao, S. Kumar, C.-H. Chen, and B. Mutnury, “PCIe Gen-5 Design Challenges of High-Speed Servers,” in *Proc. IEEE Conf. on Electrical Performance of Electronic Packaging and Systems (EPEPS)*, 2020, pp. 1–3. DOI: 10.1109/EPEPS48591.2020.9231458.
- [7] Y. S. Shao, S. L. Xi, V. Srinivasan, G.-Y. Wei, and D. Brooks, “Co-designing accelerators and SoC interfaces using gem5-Aladdin,” in *Proc. 49th Annu. IEEE/ACM Int. Symp. on Microarchitecture (MICRO)*, Taipei, Taiwan, 2016, pp. 1–12. DOI: 10.1109/MICRO.2016.7783751.
- [8] S. Rogers, J. Slycord, M. Baharani, and H. Tabkhi, “gem5-SALAM: A System Architecture for LLVM-based Accelerator Modeling,” in *Proc. 53rd Annu. IEEE/ACM Int. Symp. on Microarchitecture (MICRO)*, 2020, pp. 471–482. DOI: 10.1109/MICRO50266.2020.00047.
- [9] A. Armejach, G. López-Paradís, and M. Moretó, “gem5 + RTL: A Framework to Enable RTL Models Inside a Full-System Simulator,” in *Proc. 50th Int. Conf. on Parallel Processing (ICPP)*, 2021, pp. 1–11. DOI: 10.1145/3472456.3472461.
- [10] Y. M. Qureshi, W. A. Simon, M. Zapater, K. Olcoz, and D. Aienza, “Gem5-X: A Many-Core Heterogeneous Simulation Platform for Architectural Exploration and Optimization,” *ACM Trans. Archit. Code Optim.*, vol. 18, no. 4, Art. 44, Dec. 2021, 27 pages. DOI: 10.1145/3461662.
- [11] A. A. Khan, J. P. C. De Lima, H. Farzaneh, and J. Castrillon, “The Landscape of Compute-near-memory and Compute-in-memory: A Research and Commercial Overview,” *arXiv:2401.14428*, 2024.
- [12] J. Whitham and N. Audsley, “Studying the Applicability of the Scratchpad Memory Management Unit,” in *Proc. IEEE Real-Time and Embedded Technology and Applications Symp. (RTAS)*, 2010, pp. 205–214. DOI: 10.1109/RTAS.2010.21.
- [13] K. Paraskevas, K. Iordanou, M. Luján, and J. Goodacre, “Analysis of the Usage Models of System Memory Management Unit in Accelerator-Attached Translation Units,” in *Proc. Int. Symp. on Memory Systems (MEMSYS)*, 2020, pp. 86–96. DOI: 10.1145/3422575.3422781.
- [14] Y. Chi, X. Lin, and X. Zheng, “Design of a High-Performance SoC Simulation Model Based on Verilator,” in *Proc. 5th Int. Conf. on Algorithms, Computing and Artificial Intelligence (ACAI)*, 2022, pp. 1–6. DOI: 10.1145/3579654.3579751.
- [15] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, “DRAMsim3: A Cycle-Accurate, Thermal-Capable DRAM Simulator,” *IEEE Comput. Archit. Lett.*, vol. 19, no. 2, pp. 106–109, Jul.–Dec. 2020. DOI: 10.1109/LCA.2020.2973991.
- [16] H. Luo, Y. C. Tuğrul, F. N. Bostancı, A. Olgun, A. G. Yağlıkçı, and O. Mutlu, “Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator,” *arXiv:2308.11030v2*, 2023.
- [17] L. Steiner, M. Jung, F. S. Prado, K. Bykov, and N. Wehn, “DRAMSys4.0: An Open-Source Simulation Framework for In-Depth DRAM Analyses,” *Int. J. Parallel Program.*, vol. 50, no. 2, pp. 217–242, Apr. 2022. DOI: 10.1007/s10766-022-00727-4.
- [18] Q. Liu, M. Zapater, and D. Aienza, “MatrixFlow: System-Accelerator Co-Design for High-Performance Transformer Applications,” *arXiv preprint arXiv:2503.05290*, 2025.
- [19] A. Ivanov, N. Dryden, T. Ben-Nun, S. Li, and T. Hoefler, “Data Movement Is All You Need: A Case Study on Optimizing Transformers,” *arXiv:2007.00072*, 2020.
- [20] R. Karami, S.-C. Kao, and H. Kwon, “NonGEMM Bench: Understanding the Performance Horizon of the Latest ML Workloads with NonGEMM Operations,” *arXiv:2404.11788v3*, 2024.